

Projeto e Análise de Algoritmos I

Aula 11 - Aplicações de Busca em Grafos

Lucas Nunes Alegre

Universidade Federal do Rio Grande do Sul
Instituto de Informática
Departamento de Informática Teórica



Estes slides utilizam conteúdo adaptado da bibliografia da disciplina e também de notas de aula e slides prévios dos professores Bruno Grisci, Rodrigo Machado, André Grahl Pereira, Lucas Nunes Alegre, Marcus Ritt e Luciana Buriol.

Aplicações de Busca em Grafos Simples e Dígrafos

Busca em Grafos: Aplicações

Diversos problemas elementares em **grafos simples** e **dígrafos** podem ser **eficientemente** resolvidos utilizando buscas (**DFS** ou **BFS**).

Exemplos:

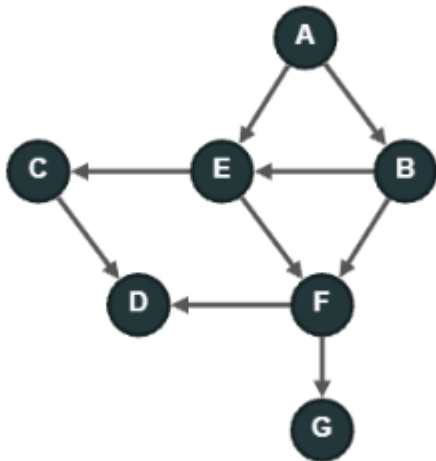
- Geração da **árvore** associada à busca
- Cálculo de **distância**
- Cálculo de **rota** entre nós
- Determinar **componentes conexos** em grafos simples
- Determinar se um grafo simples é **bipartido**
- Determinar **ordenamento topológico** em dígrafos acíclicos
- Determinar **componentes fortemente conexos** em dígrafos
- Determinar a existência de **ciclos**

Árvore de Busca

Árvore Associada à Busca

Árvore de Busca. Dada uma busca (DFS ou BFS) em G a partir de um nó v , construir uma **árvore** associada a essa busca onde **cada nó aponta para o elemento que o alcançou** diretamente.

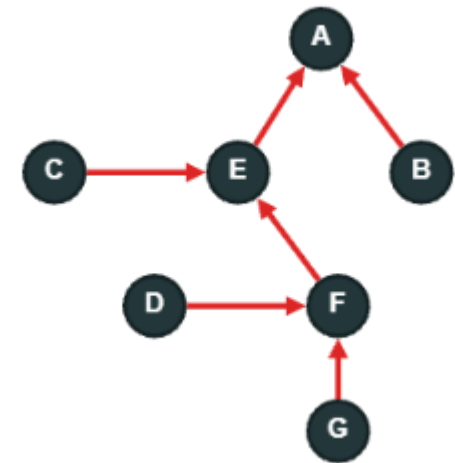
Dígrafo g_1 :



`dfs_iter(g1,
'A'):`

```
1 A
2 E
3 F
4 G
5 D
6 C
7 B
```

Árvore de busca:



- Isso é facilmente implementado com um dicionário auxiliar `parent`, preenchido ao inserirmos um nó na estrutura.
- A raiz da árvore é o nó de origem.

Arvore de Busca (DFS Iterativa)

Python Code

↺ Start Over

▶ Run Code

```
1_v def dfs_iter_tree(g, v):
2     parent = {i : i for i in g.keys()} # arvore de antecessor
3     visited = {i : False for i in g.keys()} # vetor de visitados
4
5     p = [v]
6     visited[v] = True
7
8_v     while len(p) > 0:
9         h = p.pop(0)
10_v        for i in g[h]:
11_v            if not visited[i]:
12                p.insert(0, i)
13                parent[i] = h # marca h como antecessor de i
14                visited[i] = True
15
16     return parent # devolve a arvore
17
18 # Grafo g1 do slide anterior:
19_v g1 = {
20     'A': ['B', 'E'],
21     'B': ['E', 'F'],
22     'C': ['D'],
23     'D': [],
24     'E': ['C', 'F'],
```

```
25     'F': ['D', 'G'],
26     'G': []
27 }
28
```

Cálculo de Distância em Grafo Não-Valorado

Cálculo de Distância

Cálculo de Distância. Dado um grafo simples ou dígrafo G e um nodo v , determinar um **vetor de distâncias** de v para todos os nodos de G .

BFS já fornece a distância:

- Se um dado nodo x é alcançável a partir de v , ele vai aparecer em algum dos níveis: o **nível é a respectiva distância**.
- Se não for alcançável, a distância é **infinita**.

Implementação:

- Inicia-se um vetor com valor 0 em v (e **infinito** nos demais nodos)
- Ao inserir um nodo na lista, atribui a este a distância do nodo pai (que o inseriu) + 1.

Observações:

Distância em Grafos e Digrafos

Python Code

↺ Start Over

▶ Run Code

```
1 from collections import deque
2
3 def distancia_bfs(g, v):
4     dist = {i: -1 for i in g.keys()} # distancias (-1 = infinito)
5     dist[v] = 0 # inicializa distancia da origem
6
7     visited = {i: False for i in g.keys()}
8     visited[v] = True
9
10    queue = deque([v])
11    while len(queue) > 0:
12        h = queue.popleft()
13        for i in g[h]:
14            if not visited[i]:
15                queue.append(i)
16                dist[i] = dist[h] + 1 # determina dist[i]
17                visited[i] = True
18
19    return dist # devolve o vetor de distancias
20
21 # Grafo g1:
22 g1 = {
23     'A': ['B', 'E'],
24     'B': ['E', 'F'],
```

```
25     'C': ['D'],
26     'D': [],
27     'E': ['C', 'F'],
28     'F': ['D', 'G'],
29     'G': []
30 }
31
32 print("Distâncias a partir de 'A':")
```

Outras Variações de Rota e Distância

Conectividade. Dado um grafo simples ou dígrafo G e dois nodos a e b , determinar se a está **conectado** a b .

Solução: Iniciar busca (DFS ou BFS) em a , comparando o nodo atual com b . Se achar, retorna True. Se a busca terminar sem achar b , retorna False.

Rota entre Nodos. Dado um grafo simples ou dígrafo G e dois nodos a e b , **determinar uma rota** iniciando em a e terminando em b .

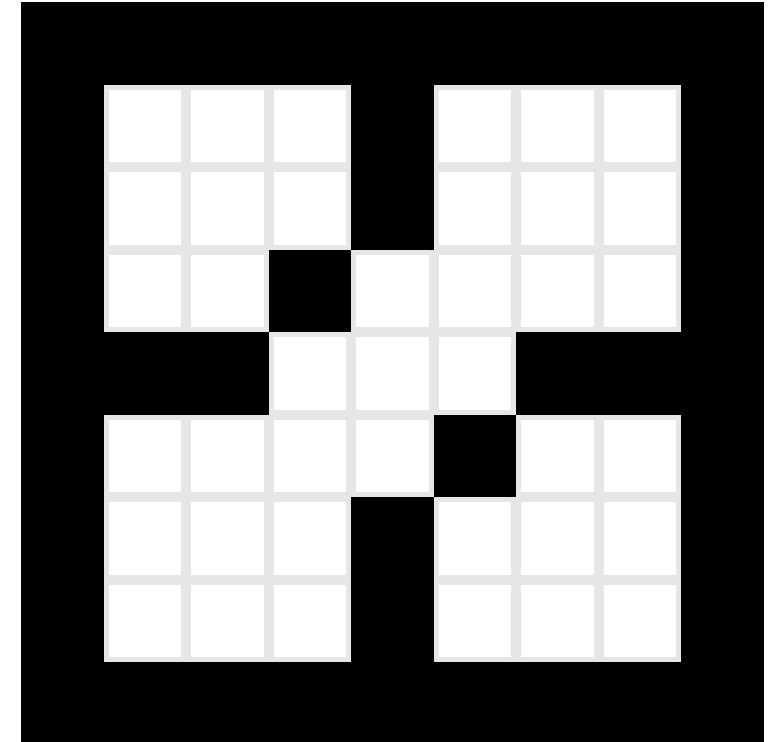
Solução: Iniciar busca (DFS ou BFS) em a produzindo a **árvore de busca**. Ao achar b , a rota entre a e b é a inversão do caminho de b até a na árvore de busca.

Rota com Menor Número de Arestas. Dado um grafo simples ou dígrafo G e dois nodos a e b , determinar uma rota com o menor número de arestas entre a e b .

Componentes Conexos

Aplicação: Flood Fill (Paint)

- Uma aplicação prática comum baseada em detecção de componentes conexos é a ferramenta “balde de tinta” ou **Flood Fill**.
- No modelo de grafos para esse problema:
 - **Nodos:** os pixels de uma imagem.
 - **Arestas:** conectam pixels adjacentes que possuem a **mesma cor**.
 - **Componente Conexo:** corresponde a uma determinada região/área da imagem de cor homogênea contígua.
- O algoritmo busca a partir de um vértice e recolora toda a região percorrida.



Componentes Conexos em Grafos Simples (UCC)

Componentes Conexos (UCC). Dado um grafo simples G , determinar o número de **componentes conexos** e quais os nós contidos em cada um (*Undirected Connected Components*).

- Como as arestas em *grafos simples* são **bidirecionais**, basta iniciar em um nó a e coletar todos os nós alcançáveis a partir dele para obter o componente conexo que contém a .
- Podemos usar um *vetor de componentes* para marcar a qual componente cada nodo pertence.
- Para calcular **todos** os componentes de um grafo:
 - Escolhemos um nó arbitrário e por meio de busca determinamos o componente ao qual pertence.
 - Após, escolhemos outro nó e repetimos até que todos os nós do grafo tenham sido percorridos.

Componentes Conexos em Grafos Simples (UCC)

Python Code

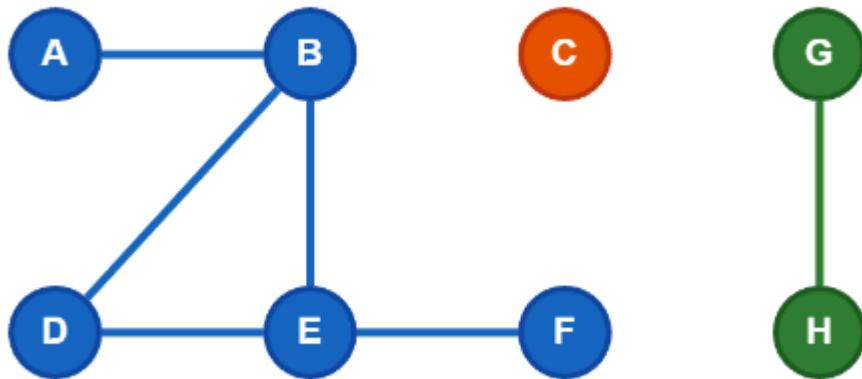
↺ Start Over

▶ Run Code

```
1_v def dfs(grafo, v, visitados, c_id, componentes):
2     visitados[v] = True
3     componentes[c_id].append(v)
4
5_v     for vizinho in grafo[v]:
6_v         if not visitados[vizinho]:
7             dfs(grafo, vizinho, visitados, c_id, componentes)
8
9_v def componentes_conexos(grafo):
10     visitados = {v: False for v in grafo}
11     componentes = {} # Mapeia id do componente para nodos
12     c_id = 0 # Começamos preenchendo o componente 0
13
14_v     for vertice in grafo:
15_v         if not visitados[vertice]:
16             componentes[c_id] = [] # Inicia novo componente
17             dfs(grafo, vertice, visitados, c_id, componentes)
18             c_id += 1 # Incrementa número de componentes
19
20     return componentes
21
22 # Exemplo com grafo simples:
```

```
23 v g_exemplo = {  
24     1: [2], 2: [1],  
25     3: [4], 4: [3, 5], 5: [4],  
26     6: []  
27 }  
28 print("Componentes conexos:")  
29 print(componentes_conexos(g_exemplo))
```

Componentes Conexos em Grafos: Exemplo



Python Code

↺ Start Over

▶ Run Code

```
1 # Grafo G2 ilustrado ao lado:
2 v g2 = {
3     'A': ['B'],
4     'B': ['A', 'D', 'E'],
5     'C': [],
6     'D': ['B', 'E'],
7     'E': ['B', 'D', 'F'],
8     'F': ['E'],
9     'G': ['H'],
10    'H': ['G'],
11 }
12
13 v def dfs(grafo, v, visitados, c_id, componentes)
14     visitados[v] = True
15     componentes[c_id].append(v)
16 v     for vizinho in grafo[v]:
17 v         if not visitados[vizinho]:
18             dfs(grafo, vizinho, visitados, c_id)
19
20 v def componentes_conexos(grafo):
21     visitados = {v: False for v in grafo}
22     componentes = {}
23     c_id = 0
24 v     for vertice in grafo:
```

```
25     if not visitados[vertice]:  
26         componentes[c_id] = []  
27         dfs(grafo, vertice, visitados, c_id)  
28         c_id += 1  
29     return componentes  
30  
31 print(componentes_conexos(g2))
```

Teste de Bipartição

Teste de Bipartição

Teste de Bipartição. Dado um grafo simples G , determinar se G é **bipartido** (isto é, se seus vértices podem ser particionados em dois conjuntos independentes V_1 e V_2 , ou equivalentemente, se **não possui ciclos ímpares**).

Conforme visto, um grafo é bipartido se e somente se não possuir ciclos ímpares. Esta caracterização nos fornece um algoritmo bastante eficiente baseado em **coloração de nodos** e BFS para testar bipartição.

- Defina um vetor `cor` que associa **true** (cor 1) ou **false** (cor 0) a cada nodo.
- Para cada componente do grafo, escolha um nodo e pinte com cor 1. Inicie uma busca em largura na qual todos os nodos alcançáveis a partir de um dado nodo precisam ser pintados com a cor oposta.

Teste de Bipartição: Código

Python Code

↺ Start Over

▶ Run Code

```
1 from collections import deque
2
3 def componente_bipartido(g, v, cor, visited):
4     queue = deque([v])
5     cor[v] = 1 # atribui cor 1 a v
6     visited[v] = True # marca v como visitado
7     while len(queue) > 0:
8         i = queue.popleft()
9         for j in g[i]:
10            if visited[j] and cor[i] == cor[j]:
11                return False # achou ciclo ímpar
12            elif not visited[j]:
13                queue.append(j)
14                visited[j] = True
15                cor[j] = 1 - cor[i] # atribui cor inversa
16        return True
17
18 def bipartite(g):
19     cor = {} # vetor de cores (1 e 0)
20     visited = {i: False for i in g.keys()}
21     for x in g.keys(): # escolhe um nodo x por componente
22         if not visited[x]:
23             if not componente_bipartido(g, x, cor, visited):
24                 return False # achou ciclo ímpar
```

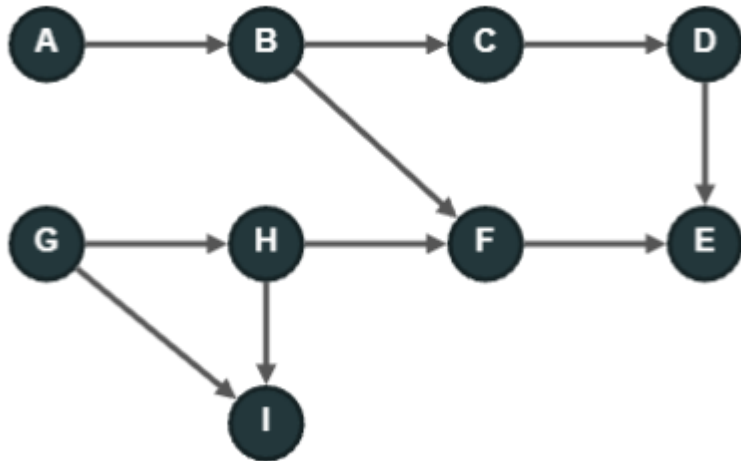
```
25
26     return True # não há ciclo ímpar
27
28 # Grafo bipartido (ciclo par C4):
29 g_bipartido = {1: [2, 4], 2: [1, 3], 3: [2, 4], 4: [1, 3]}
30 print("g_bipartido é bipartido?", bipartite(g_bipartido))
31
32 # Grafo não bipartido (triângulo C3 - ciclo ímpar):
33 g_nao_bipartido = {1: [2, 3], 2: [1, 3], 3: [1, 2]}
```

Ordenamiento Topológico

Ordenamento Topológico

Ordenamento Topológico. Dado um dígrafo acíclico (DAG) $G = (V, E)$, encontrar uma sequência linear $v_1, v_2, \dots, v_n$ contendo todos os vértices de V tal que, para toda aresta direcionada $(v_i, v_j) \in E$, o vértice origem v_i precede o vértice destino v_j na sequência (isto é, $i < j$).

Exemplo:



Alguns possíveis ordenamentos topológicos para G_5 :

- $A, G, B, H, I, C, D, F, E$
- $G, H, I, A, B, F, C, D, E$

Algoritmo de Kahn para Ordenamento Topológico

Algoritmo de Kahn (1962) para ordenamento topológico:

- Um nodo de **grau de entrada zero** é seguro para iniciar o ordenamento
- Ao remover ele do grafo (e suas arestas de saída), podem-se **gerar novos nodos de grau zero**
- Podemos pré-calcular o grau de entrada de todos os nós (armazenando em um vetor `inDeg`) e iniciar uma lista *q* contendo nodos de grau zero a serem processados
- Removemos um nodo *x* de *q* e colocamos no final do ordenamento gerado
- Após, decrementamos o grau de todos os nós alcançáveis por *x*: se o grau de um deles chegar a zero, o inserimos em *q*. Repita até esgotar *q*.

Algoritmo de Kahn para Ordenamento Topológico: Código

Python Code

↺ Start Over

▶ Run Code

```
1  from collections import deque
2
3  def toposortKahn(g):
4      inDeg = {i: 0 for i in g.keys()} # graus de entrada
5
6      for i in g.keys(): # para todo nodo i
7          for j in g[i]: # para todo j vizinho de i
8              inDeg[j] += 1 # +1 ao grau de entrada de j
9
10     q = deque([]) # lista de nodos fonte (source)
11     l = [] # lista de saida (ordenamento)
12     for i in g.keys():
13         if inDeg[i] == 0:
14             q.append(i) # inicializa q com nodos de grau 0
15
16     while len(q) > 0:
17         h = q.popleft() # extrai h de q
18         l.append(h) # insere h em l
19         for i in g[h]: # para todos vizinho i de h
20             inDeg[i] -= 1 # decrementa grau de i
21             if inDeg[i] == 0: # se grau chega a zero
22                 q.append(i) # insere na lista de nodos fonte
```

```

23
24     return l # retorna a lista de nodos
25
26 # Dígrafo acíclico G5 (do slide anterior):
27 v g5 = {
28     'A': ['B'],
29     'B': ['C', 'F'],
30     'C': ['D'],
31     'D': ['E'],
32     'E': [],
33     'F': ['E'],
34     'G': ['H', 'I'],
35     'H': ['F', 'I'],
36     'I': []
37 }
38 print("Ordenamento topológico (Kahn):")
39 print(toposortKahn(g5))

```

Ordenamento Topológico com DFS

Busca em profundidade (DFS) também pode ser utilizada para determinar ordenamento topológico.

- **Inserção no Topo:**

- Um nodo v só é inserido na lista pos após o retorno de todas as chamadas recursivas de DFS geradas por ele.
- Ao usar `pos.insert(0, v)`, garantimos que as dependências apareçam antes de seus sucessores na lista final.

- **Garantia de Cobertura:**

- O laço principal percorre todos os nodos do grafo, garantindo que mesmo componentes desconexas sejam processadas.

- **Resultado:**

- A lista final é uma ordenação linear onde, para toda aresta $(u \rightarrow v)$, o nodo u precede v .

Ordenamento Topológico (DFS, Lista de Nodos)

Python Code

↻ Start Over

▶ Run Code

```
1_v def toposort(g):
2     pos = [] # lista de nodos
3     visited = {i: False for i in g.keys()}
4
5_v     def dfsTOPO(g, v, pos, visited):
6         visited[v] = True
7_v         for i in g[v]:
8_v             if not visited[i]: # navega a partir de v
9                 dfsTOPO(g, i, pos, visited)
10        pos.insert(0, v) # insere v no topo da lista
11
12_v    for i in g.keys():
13_v        if not visited[i]: # inicia ordenamento
14            dfsTOPO(g, i, pos, visited)
15
16    return pos # retorna a lista de nodos em ordem
17
18 # Dígrafo acíclico G5:
19_v g5 = {
20     'A': ['B'],
21     'B': ['C', 'F'],
22     'C': ['D'],
23     'D': ['E'],
24     'E': [],
```

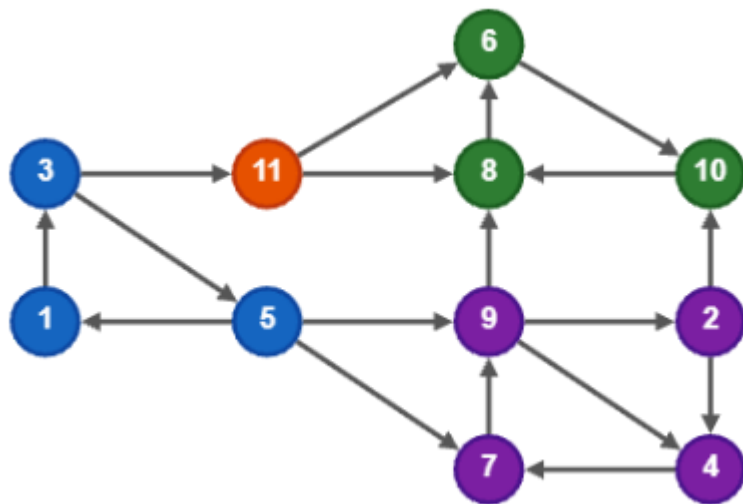
```
25     'F': ['E'],
26     'G': ['H', 'I'],
27     'H': ['F', 'I'],
28     'I': []
29 }
30 print("Ordenamento topológico (DFS):")
31
```

Componentes Fortemente Conexos

Componentes Fortemente Conexos (SCC)

Componentes Fortemente Conexos (SCC). Dado um dígrafo $G = (V, E)$, decompor V em **componentes fortemente conexos** (*Strongly Connected Components*), onde cada componente é um subdígrafo conexo maximal no qual todo par de vértices é mutuamente alcançável.

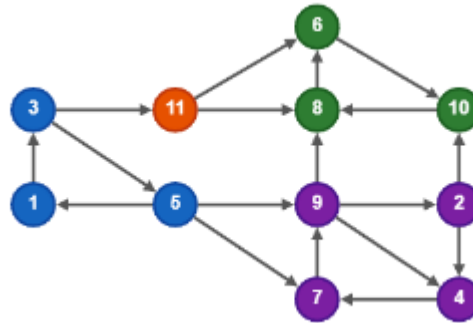
Diferentemente de grafos simples, por conta da direcionalidade dos arcos, pode haver mais de um componente mesmo que o dígrafo pareça estar “em um único pedaço”.



Componentes fortemente conexos (nodos):

- [1, 3, 5]
- [11]
- [6, 8, 10]
- [2, 4, 7, 9]

Componentes Fortemente Conexos (SCC) (Cont.)



Componentes: [1, 3, 5], [11], [6, 8, 10], [2, 4, 7, 9]

Algoritmo de Kosaraju-Shamir (SCC)

Kosaraju (e **Shamir**, independentemente) descobriram que, ao rodar **ordenamento topológico** sobre um dígrafo arbitrário (não necessariamente acíclico), temos que o primeiro nodo do ordenamento obrigatoriamente pertence a um componente fonte (*source*).

- Porém, observe que precisamos de um nodo do sumidouro, não da fonte!

Solução: toposort sobre o **grafo reverso** (G^R)! Isso gera uma sequência de nodos que pode ser usada para guiar uma busca por componentes no grafo original que nunca inclui mais do que o componente atual de cada nodo.

Algoritmo de Kosaraju-Shamir (SCC)

Python Code

↺ Start Over

▶ Run Code

```
1_v def rev(g):
2     """Retorna o grafo reverso gR com as arestas invertidas."""
3     gR = {i: [] for i in g.keys()}
4_v     for u in g:
5_v         for v in g[u]:
6             gR[v].append(u)
7     return gR
8
9_v def toposort(g):
10     pos = []
11     visited = {i: False for i in g.keys()}
12_v     def dfsTOPO(v):
13         visited[v] = True
14_v         for u in g[v]:
15_v             if not visited[u]:
16                 dfsTOPO(u)
17         pos.insert(0, v)
18_v     for i in g.keys():
19_v         if not visited[i]:
20             dfsTOPO(i)
21     return pos
22
23_v def strongComponents(g):
24     gR = rev(g) # reverte o grafo
```

```

25 order = toposort(gR) # calcula ordem de busca por gR
26 comp = {} # vetor de componentes
27 atual = 1 # componente inicial
28 visited = {i: False for i in g.keys()}
29
30 def dfsSCC(g, v, comp, atual, visited):
31     visited[v] = True
32     comp[v] = atual # registra o componente atual
33     for i in g[v]:
34         if not visited[i]:
35             dfsSCC(g, i, comp, atual, visited)
36
37 for i in order: # na ordem adequada calculada via gR
38     if not visited[i]: # busca o componente de i
39         dfsSCC(g, i, comp, atual, visited)
40         atual += 1 # incrementa componente
41
42 return comp # retorna vetor de componentes
43
44 # Dígrafo de 11 vértices dos slides anteriores:
45 g_scc = {
46     1: [3],
47     2: [4, 10],
48     3: [5, 11],
49     4: [7],
50     5: [1, 7, 9],
51     6: [10],
52     7: [9],
53     8: [6],
54     9: [2, 4, 8],
55     10: [8],

```

```
56     11: [6, 8]
57 }
58
59 print("Componentes fortemente conexos (Kosaraiu):")
```

Detecção de Ciclos

Detecção de Ciclos

Detecção de Ciclos. Dado um grafo simples, pseudografo ou dígrafo G , determinar a **existência de ciclos** (se G contém pelo menos um ciclo).

Há diferenças importantes entre os modelos com arcos unidirecionais ou bidirecionais:

- Em um **grafo simples**, os menores ciclos são **triângulos**;
- Em um **dígrafo**, um **loop** ou **um par de arcos de ida e volta** configuram ciclo;
- Em um **pseudografo**, um **loop** ou **arestas paralelas** configuram ciclo.

Detecção de Ciclos em Grafos Simples (DFS)

Ideia principal: Para cada componente conexo (nó ainda não visitado), iniciamos uma **busca em profundidade (DFS)** e mantemos o **nó pai** de cada vértice.

- Como o grafo é não-direcionado, cada aresta aparece duas vezes (uma em cada direção). Assim, ao explorar um vértice, devemos ignorar a aresta que leva de volta ao seu pai.
- Durante a busca, se encontrarmos um vizinho já visitado **que não seja o pai do vértice atual**, então detectamos um ciclo.
- Repita o processo para todos os vértices ainda não visitados (componentes conexos).

Detecção de Ciclos em Dígrafos (Kahn)

Ideia principal: Rodar o algoritmo de ordenamento topológico de Kahn sobre o dígrafo (baseado em graus de entrada).

- Se houver ciclos, isso impede que os nós do ciclo em questão obtenham **grau de entrada 0** e possam ser inseridos na solução.
- Isto é, o algoritmo vai listar um ordenamento com **menos vértices** do que o total de vértices do grafo **se o grafo contém algum ciclo**.

Conclusão

Moral da Aula

Podemos usar **DFS** e **BFS** para resolver diversos problemas em grafos simples e dígrafos de forma eficiente.

Complexidade Linear: $\Theta(|V| + |E|)$

